

In []: โจทย์ข้อที่ 1 โจทย์ข้อที่

1: การเขียนตัวแปลงสีเทาด้วยสูตรคณิตศาสตร์แบบแมนนวล (Manual Grayscale Converter)
แม้ว่า OpenCV จะมีคำสั่ง `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` แต่ในฐานะนักวิเคราะห์ข้อมูลภาพ นักศึกษาต้องเข้าใจโครงสร้างพิกเซลในแง่ of
เชิงเส้น

1. ให้อ่านไฟล์ภาพ `sample.jpg` เข้ามาในโปรแกรม

2. ทำการสกัดแยกแชนแนลสีแดง (R), เขียว (G), และน้ำเงิน (B) ออกจากกันโดยการอ้างอิงพิกัดอาร์เรย์ (Array Slicing) ของ NumPy (ห้ามใช้ฟังก์ชัน
สำเร็จรูปของ OpenCV)

3. ทำการคำนวณสร้างภาพระดับสีเทาขึ้นใหม่ด้วยตนเอง โดยประยุกต์ใช้ความรู้จากหัวข้อบรรยายเรื่องสูตรการสะท้อนความสว่างมาตรฐาน (Photopic Curve
Standard): $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$

4. แสดงภาพผลลัพธ์ที่ได้จากการแปลงแมนนวล เปรียบเทียบกับภาพที่แปลงด้วยคำสั่งสำเร็จรูปของ OpenCV เติงข้างกันโดยใช้ Matplotlib

5. บันทึกภาพระดับสีเทาที่คำนวณได้ด้วยสูตรของนักศึกษาลงในระบบไฟล์คอมพิวเตอร์ตั้งชื่อไฟล์ว่า `my_gray_output.png`

In [19]: # โจทย์ข้อที่ 1

```
# import library ที่จำเป็นต่อการใช้งาน
import cv2
import numpy as np
import matplotlib.pyplot as plt

# นำเข้ารูปภาพที่ต้องการ
img_bgr = cv2.imread("Dog.jpg")

# แปลงสีรูปจาก bgr ไปเป็น rgb
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# สกัดแชนแนลแต่ละสี
R = img_rgb[:, :, 0]
G = img_rgb[:, :, 1]
B = img_rgb[:, :, 2]

print(R, G, B)

# สูตรมาตรฐาน ITU-R BT.601 / Luminance
# Gray = 0.299 * R + 0.587 * G + 0.114 * B

# เนื่องจากค่าสีมีค่าเป็น int หากจะมาคำนวณกับสูตรได้ ต้องแปลงค่าเป็น float เพื่อกัน error
R_f = R.astype(np.float64)
G_f = G.astype(np.float64)
B_f = B.astype(np.float64)

# แทนค่าลงในสูตร
gray_manual = 0.299 * R_f + 0.587 * G_f + 0.114 * B_f

# แปลงค่าที่ได้จากการคำนวณเป็นชนิด int
gray_manual = np.clip(gray_manual, 0, 255).astype(np.uint8)

# สร้างภาพ gray จาก opencv เพื่อนำมาเปรียบเทียบ
gray_cv2 = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

fig, axes = plt.subplots(1, 2, figsize = (12, 5))

axes[0].imshow(gray_manual, cmap='gray')
axes[0].set_title('Manual Gray\n0.299R + 0.587G + 0.114B')
axes[0].axis('off')

axes[1].imshow(gray_cv2, cmap='gray')
axes[1].set_title('OpenCV cvtColor\n(BGR2GRAY)')
axes[1].axis('off')

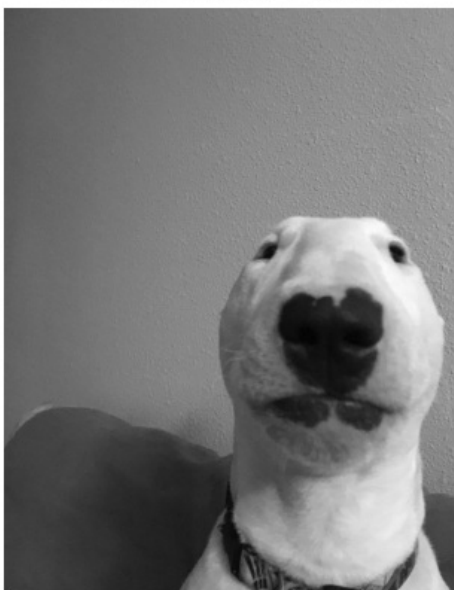
plt.suptitle('Grayscale Comparison', fontsize=14)
plt.tight_layout()
plt.savefig('comparison.png', dpi=150)
plt.show()

# บันทึกไฟล์ผลลัพธ์
cv2.imwrite('my_gray_output.png', gray_manual)
print("บันทึก my_gray_output.png เรียบร้อยแล้ว")
```

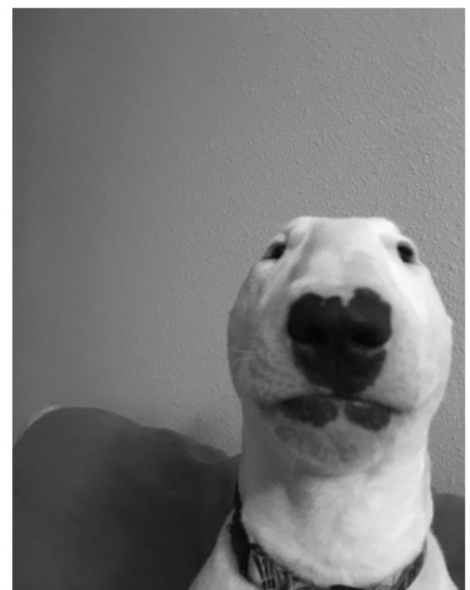
```
[[ 83  61  75 ... 129 130 137]
 [ 83  76  88 ... 123 123 127]
 [ 67  67  82 ... 121 119 122]
 ...
 [ 58  62  62 ...  71  67  72]
 [ 57  62  62 ...  66  61  66]
 [ 61  65  67 ...  62  57  63]] [[ 86  64  78 ... 132 133 140]
 [ 86  79  91 ... 126 126 130]
 [ 70  70  85 ... 124 122 125]
 ...
 [ 47  51  53 ...  62  58  63]
 [ 46  51  53 ...  57  52  57]
 [ 50  54  58 ...  53  48  54]] [[ 67  45  59 ... 111 112 119]
 [ 67  60  72 ... 105 105 109]
 [ 53  53  68 ... 105 103 106]
 ...
 [ 45  49  48 ...  55  51  56]
 [ 44  49  48 ...  50  45  50]
 [ 48  52  53 ...  46  41  47]]
```

Grayscale Comparison

Manual Gray
0.299R + 0.587G + 0.114B



OpenCV cvtColor
(BGR2GRAY)



บันทึก my_gray_output.png เรียบร้อยแล้ว

In []: อภิปรายว่าเมื่อนำค่าความสว่างที่แปลงด้วยสูตร $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ ไปเทียบกับฟังก์ชันสำเร็จรูปของ OpenCV แล้ว ผลลัพธ์มีความแตกต่างกันในระดับสายตาและในเชิงค่าตัวเลขข้อมูลพิกเซลหรือไม่อย่างไร?

ตอบ ไม่เห็นความแตกต่างระดับสายตา แต่ระดับเชิงค่าตัวเลขต่างกันเล็กน้อย +- ไม่เกิน 1-2 พิกเซล

In []: โจทย์ข้อที่2: การคัดเลือกและสกัดพื้นที่ภาพเฉพาะจุด (Region of Interest - ROI Extraction)

ในหลายกรณีของการวิเคราะห์ภาพ เราไม่ได้ประมวลผลทั้งภาพแต่จะเลือกสกัดเฉพาะจุดที่ต้องการตรวจสอบ

1. ให้อ่านพารามิเตอร์ขนาดภาพ (shape) จากนั้นระบุพิกัดที่ตั้งกรอบสี่เหลี่ยมขึ้นมาเพื่อเลือกสกัดพื้นที่ที่ใจกลางภาพ (Center of the Image) ขนาดความกว้าง พิกเซล และความสูง พิกเซล
2. ทำการสกัดภาพส่วนย่อย (Cropping) จากตำแหน่งพิกัดใจกลางที่คำนวณได้ออกมาเป็นตัวแปรภาพใหม่
3. ใช้คำสั่งวาดภาพเรขาคณิตของ OpenCV เพื่อทำการติกรอบสี่เหลี่ยมสีแดง (เส้นหนา 3 พิกเซล) ทับลงบนภาพต้นฉบับ ณ ตำแหน่งที่ . สกัดภาพย่อยออกไป
4. จัดเก็บภาพย่อยที่สกัดมาได้ในชื่อ roi_center.jpg และภาพต้นฉบับที่ติกรอบสีแดงแล้วในชื่อ sample_marked.jpg

In [3]: # โจทย์ข้อที่ 2

```
# import library ที่จำเป็นต่อการใช้งาน
import cv2
import numpy as np
import matplotlib.pyplot as plt

# นำเข้ารูปภาพที่ต้องการ
img_bgr = cv2.imread("Dog.jpg")

# อ่าน shape เพื่อนำมาคำนวณหาใจกลางภาพ
h, w, c = img_bgr.shape
print(f"ขนาดภาพ: {w}x{h} pixels, {c} channels")

# กำหนดค่า roi ตามโจทย์
roi_w = 200 # ความกว้าง
```

```

roi_h = 200 # ความสูง

# คำนวณพิกัดมุมบนซ้าย (x1, y1) และมุมล่างขวา (x2, y2)
x1 = (w - roi_w) // 2
y1 = (h - roi_h) // 2
x2 = x1 + roi_w
y2 = y1 + roi_h

print(f"พิกัด ROI: ({x1}, {y1}) → ({x2}, {y2})")

# (y1:y2 เลือก rows แถว y บน ล่าง) (x1:x2 เลือก cols แถว x ซ้าย ขวา)
roi = img_bgr[y1:y2, x1:x2]

print(f"ขนาด ROI ที่ได้: {roi.shape}")

# เตรียมวาดกรอบสีแดงทับลงรูปเดิม
img_marked = img_bgr.copy()

# สร้างกรอบสีแดง
cv2.rectangle(
    img_marked,
    (x1, y1),      # มุมบนซ้าย
    (x2, y2),      # มุมล่างขวา
    (0, 0, 255),   # สีแดง (BGR)
    3              # ความหนาเส้น 3 px
)

# บันทึกไฟล์
cv2.imwrite('roi_center.jpg', roi)
cv2.imwrite('sample_marked.jpg', img_marked)
print("บันทึกไฟล์เรียบร้อยแล้ว: roi_center.jpg, sample_marked.jpg")

# แสดงผลเปรียบเทียบ
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].imshow(cv2.cvtColor(img_marked, cv2.COLOR_BGR2RGB))
axes[0].set_title(f'Original + Red Rectangle\n({w}x{h})')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
axes[1].set_title(f'ROI Crop\n({roi_w}x{roi_h})')
axes[1].axis('off')

plt.tight_layout()
plt.show()

```

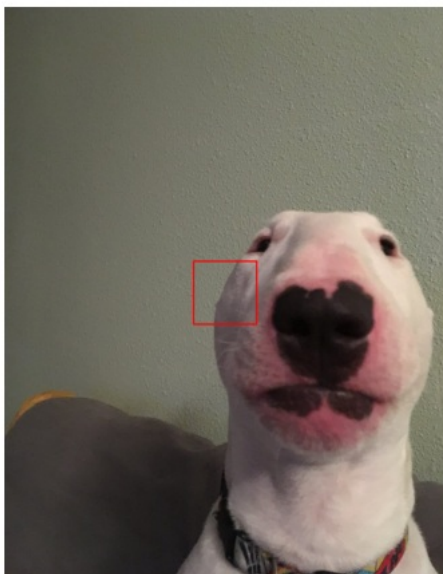
ขนาดภาพ: 1400x1815 pixels, 3 channels

พิกัด ROI: (600, 807) → (800, 1007)

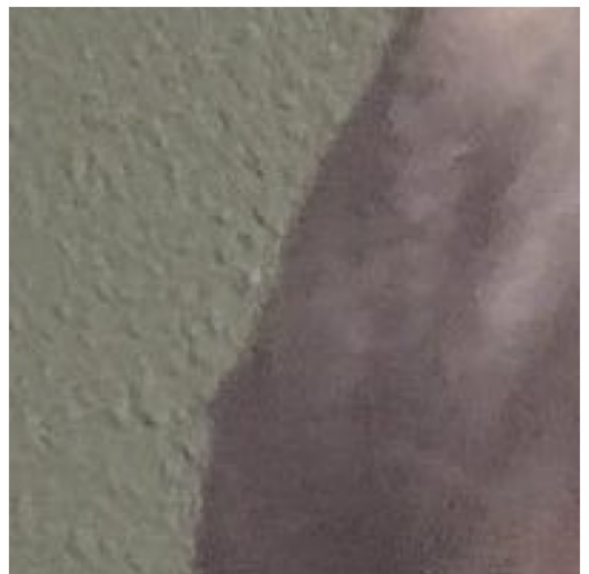
ขนาด ROI ที่ได้: (200, 200, 3)

บันทึกไฟล์เรียบร้อยแล้ว: roi_center.jpg, sample_marked.jpg

Original + Red Rectangle
(1400x1815)



ROI Crop
(200x200)



In []: อธิบายว่าการสับเปลี่ยนสัญญาณและแกนสีจาก BGR เป็น RGB ส่งผลกระทบอย่างไรต่องานประเภทสกัดวัตถุ⁴ มีสีเฉพาะเจาะจง?

ตอบ การสับเปลี่ยนสัญญาณและแกนสีจาก BGR เป็น RGB ส่งผลกระทบต่องานสกัดวัตถุที่มีสีเฉพาะเจาะจงอย่างมีนัยสำคัญ เนื่องจาก OpenCV อ่านภาพในรูปแบบ BGR หากผู้ใช้อ้างอิง index แกนสีโดยไม่แปลงรูปแบบก่อน จะทำให้การ threshold ผิดแกน

